# Optimus Primal — Gait Optimization

## From Hand-Tuned Crawl to Learned Locomotion

Darien Parris

April 28, 2026

# Problem

- **Slow iteration loop** — each hand-tuned gait takes minutes to test on hardware; no way to explore the full parameter space
- **Stride length hit a stability ceiling** — longer strides toppled the robot; couldn't find the right weight-shift timing by hand
- **Discrete hand-scripted phases don't generalize** — moving to simulation lets the robot optimize against *intrinsic* goals (keep the frame parallel to the ground) rather than fixed motor sequences

# Digital Twin — URDF to MuJoCo

- Built from the same `optimus_primal.urdf` the real robot uses — same link geometry, joint limits, effort caps
- 8 actuated joints (4 hips + 4 knees) driven by **position servos** ($k_p$=2.5, $k_v$=0.05, initial guess; bench-measured later — see slide 13)
- Free joint on `base_link` — physics decides what happens; no kinematic cheating
- Floor: flat plane with friction $\mu$=1.0 (rubber/TPU-sock approximation)
- **Caveat:** CAD wasn't modeled at the origin, so STL meshes render detached. Collision geometry is correct; visual hidden via `discardvisual`. Cosmetic only — physics unaffected.

# Search Space — 105 Parameters

- Gait **structure is fixed**: 13 phases in a crawl cycle
  `start` $\rightarrow$ {`shift, swing, plant`} $\times$ {`FR, RL, FL, RR`}
  Optimizer tunes the **angles at each phase**, not the sequencing
- **104 joint-angle params:** 13 phases $\times$ 4 legs $\times$ 2 joints (hip + knee), in degrees
- **+1 phase-time param:** seconds between phases — controls gait cadence

| Parameter | Lower Bound | Upper Bound |
|---|---|---|
| Hip angle | $5^\circ$ | $55^\circ$ |
| Knee angle | $-100^\circ$ | $-25^\circ$ |
| Phase time | 0.15 s | 1.2 s |

- **Seeded from the hand-tuned gait** — generation 1 already walks
- **Sim-to-real is a parameter copy** — same degree convention as the robot's motor controller; winning JSON loads directly onto hardware

# Algorithm — CMA-ES

**Covariance Matrix Adaptation Evolution Strategy**

Gradient-free, population-based optimizer — the standard for robot gait tuning.

**Each generation:**

1. Sample 48 candidate gaits from a multivariate Gaussian $\mathcal{N}(\mathbf{m}, \sigma^2\mathbf{C})$
2. Evaluate every candidate in MuJoCo — in parallel across CPU cores
3. Rank by reward, keep the top half
4. Update the distribution:
   - **Mean m** moves toward the winning region
   - **Covariance C** stretches along directions that produced good solutions
   - **Step size** $\sigma$ grows or shrinks based on progress

**Engineering:** popsize $= 48$, $\sigma_0 = 5.0$, auto-restart on 15-gen stall, parallel evaluation ($\sim 7\times$ speedup on 8-core Mac), checkpoint every generation.

# Reward Function

**Shaped iteratively through failure.**

| Component | Purpose | Weight |
|---|---|---|
| Forward distance | Primary goal | $+100/\mathrm{m}$ |
| Phases completed | Survival bonus | $+0.5/\mathrm{phase}$ |
| Mean body height | Don't collapse | $+5$ |
| Backward distance | Wrong direction | $-200/\mathrm{m}$ |
| Mean pitch$^2$ | Stay level (F/B) | $-30$ |
| Mean roll$^2$ | Stay level (L/R) | $-40$ |
| Peak pitch/roll$^2$ | No big momentary tilts | $-8$ each |
| **Mean nose-down$^2$** | **Don't flop forward** | $\mathbf{-200}$ |
| **Peak nose-down$^2$** | **Not even once** | $\mathbf{-60}$ |
| Mean pitch-rate$^2$ | No floppy motions | $-2$ |

**Hard terminations:** body height $< 8\,\mathrm{cm}$, any tilt $> 30^\circ$, nose-down $> 27^\circ$.

# Reward Function — Iteration Story

**Each version was shaped by watching what the optimizer exploited.**

v1 — Distance only Robot learned to collapse onto its shins and scoot forward. Technically moved. Not a gait.

v2 — Symmetric tilt penalty Tightened fall threshold from 45° to 30°. Better balance, but still toppled forward during swing phases.

v3 — Directional nose-down + pitch-rate Targets the specific forward-flop failure mode without penalizing backward lean (useful for weight shifting).

*The reward function is as much the product of learning as the gait itself.*

# Results — Does the Starting Point Matter?

Three CMA-ES runs, same reward function, different seeds:

**Hand-tuned gait** Seeded from the manually-tuned `stance.py` crawl

**Random** Uniform random within bounds

**Neutral standing** Every phase pinned to the standing pose (no motion)

**Takeaway:** Seeding from a human-tuned gait gives a massive head start — generation 1 is already walking, so the optimizer spends its budget *refining* instead of *discovering*.

# Results — Algorithm Comparison

Same reward function, same seed, different optimization algorithms:

**CMA-ES** Population-based, learns covariance structure of the search space

**Differential Evolution** Population-based, mutates by combining existing solutions

**Random Search** Uniform sampling baseline — "does optimization even help?"

# Limitation — Open-Loop Control

**CMA-ES found the best choreography. But it's still a choreography.**

- Output is a **fixed lookup table**: 13 phases, 8 angles each, played back identically every stride
- No sensors, no feedback, no ability to react
- Works in sim (perfect flat floor) but struggles on real hardware

| | CMA-ES (current) | What we need |
|---|---|---|
| Output | 105 fixed numbers | A function that reacts |
| Sensors at runtime | None — blind playback | IMU, joint positions |
| Adapts to disturbance | No | Yes |
| What it learned | "The best script" | "How to walk" |

# Next — Reinforcement Learning

**Close the loop: from choreography to skill.**

RL trains a neural network (a "policy") that takes sensor readings *every timestep* and outputs motor commands:

$$\texttt{observe} \rightarrow \texttt{decide} \rightarrow \texttt{act} \rightarrow \texttt{observe} \rightarrow \texttt{decide} \rightarrow \texttt{act} \rightarrow \ldots$$

**What we reuse:**

- The MuJoCo digital twin — same sim
- The reward function — same objectives, already well-shaped
- The CMA-ES gait — baseline to beat, can seed the initial policy

**What's new:**

- Observation space: IMU (pitch, roll, $\dot{\theta}$) + 8 joint positions + 8 joint velocities
- Algorithm: PPO (Proximal Policy Optimization)
- Output: a neural network, not a JSON of fixed angles

*CMA-ES found the best choreography for a perfect floor.*
*RL will learn to actually walk — adjusting every step based on what the robot feels.*

# PPO — Reward Engineering Journey (v5–v34)

**30+ reward variants. Learning the failure modes was the work.**

| Era | What we tried | What broke |
|---|---|---|
| v5–v14 | Distance + tilt + height + extension | "Reward-engineering ceiling" — combined ∼13 |
| v15–v18 | Stride-detection bonuses (**silent bug**) | Foot-name lookup returned –1; rewards never fire |
| v20 | Post-fix recipe | Combined ∼42; new champion |
| v23–v26 | Body-smoothness + foot-drift penalties | Body smoothness helped (N=2, noisy) |
| v27–v29 | Survival bonus, fall-penalty schedules | Most regressed; high seed variance |
| v30 | **Friction domain randomization** | *Best sim result, doesn't deploy* |
| v31 | High kp ($k_p$=20) for deployable PD | Policy never learned to walk |
| v33–v34 | X-config + low-friction + moderate steps | Same actuator-gap, 70–82% bang-bang |

**High-water marks:** v20 (combined ∼42), v25_scratch_3M_b (44.7), v26_body_smooth_b (44.9).

*Across 23 trials in 9 cohorts, within-cohort seed variance (±10–30) dominated between-cohort effects.*

PPO learning curves — selected runs

# Sim-to-Real — Friction Domain Randomization (v30)

**Real surfaces aren't sim-default ($\mu$=1.0). Trained three variants over $\mu \in [0.3, 1.0]$.**

| | $\mu$=1.0 | $\mu$=0.7 | $\mu$=0.5 | $\mu$=0.3 | $\Delta(1.0 \to 0.3)$ |
|---|---|---|---|---|---|
| **bodysmooth_dr** | 50.5 m | 56.4 m | 54.0 m | **53.9 m** | **+7%** |
| v20_dr | **61.2 m** | 56.5 m | 51.5 m | 49.4 m | −19% |
| v6trig_dr | 35.1 m | 37.1 m | 35.1 m | 39.6 m | +13% |

**Finding:** `bodysmooth_dr` is *flat across the friction range* — exactly what DR was supposed to produce. Pick for any low-friction deploy surface.

*But* — preparing this for hardware uncovered the **actuator gap**, a sim-to-real failure mode unrelated to friction.

# The Actuator Gap — Why PPO Doesn't Deploy

**Setup:** Trying to load the v30 friction-DR champion onto hardware revealed something more fundamental than friction.

**Diagnostic measurements:**

1. **Bang-bang action saturation: 88%**
   Almost every commanded joint target is at the action-space LOW or HIGH.
2. **Joint qpos response oscillates hard**
   Under MuJoCo's PD ($k_p$=2.5, $k_v$=0.05), the joint slowly chases the bang-bang target. P95 $|\Delta q|$/step $= 17°$.
3. **Open-loop replay test: 65.8 m → 0.57 m**
   Take the joint trajectory the body actually followed, feed it as the env's target sequence in a fresh deterministic rollout. The body doesn't walk anymore.

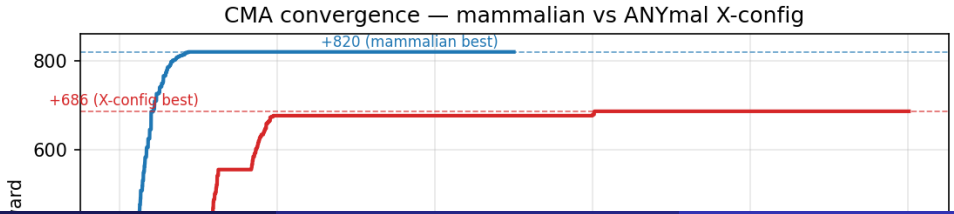
**Interpretation:** *Walking lives in the bang-bang action stream interacting with the PD, not in the resulting joint trajectory.* PPO discovered the underdamped PD acts as a low-pass filter; saturated commands at the right phase produce useful body motion. Same trick can't reproduce on the LX-16A's trajectory generator.

# ANYmal X-Config — Geometry Experiment

**Hypothesis:** An X-stance (front knees forward, rear hips swept forward, all four feet under chassis) might be more stable to walk on than the mammalian default.

- Hand-posed the robot kinesthetically; captured per-leg servo angles
- Built `optimus_primal_xconfig.urdf` with rear hip + rear knee axes flipped
- Hardware `--xconfig` flag mirrors the same flip in `leg_abs`
- Visual sanity-check via `view_xconfig.py` — chassis stands level, four feet planted

**X-config CMA:** climbed cleanly to **best** $\approx +686$ by gen $\sim$300 ($\sim$84% of the mammalian gen-2500 winner at $\frac{1}{5}$ the compute).



# Closing the Sim-to-Real Loop — Bench Calibration

**Measure the actual LX-16A response, fit MuJoCo PD parameters to match.**

`robot/motor_step_response.py` on the Pi:

1. Step each of 8 servos $\pm 10^\circ$ from current position
2. Sample `get_physical_angle()` at 50 Hz for 300 ms each direction
3. Save the time-series as JSON

`sim/fit_pd_from_step_response.py` on the Mac: sweeps MuJoCo $(k_p, k_v)$, finds best fit per servo.

**Result:**

| Joint type | $k_p$ | $k_v$ | RMSE |
|---|---|---|---|
| Front hips | 0.47 | 1.23 | 6.0° |
| Rear hips | 1.14 | 1.92 | 5.4° |
| All knees | 0.73 | 0.5–0.9 | 7.0° |
| **Median** | **0.73** | **0.87** | — |

**Two conclusions:**

1. Original $k_p$=2.5 was *too stiff* — sim was a faster actuator than reality.

# The Fix — Velocity-Slew Actuator

**Don't tune the gains. Change the actuator model.**

Added `slew_rate_dps` parameter to `OptimusPrimalEnv.step()`: per-substep update to `data.ctrl` is clamped to `slew_rate` $\times$ `dt` of change toward the policy's action. Emulates the LX-16A's internal constant-velocity trajectory generator at the actuator level.

**Verified:** a bang-bang action stream now produces only $\sim 0.6^\circ$/env-step of joint motion at $125^\circ$/s slew rate (vs the unconstrained PD's $\pm 17^\circ$/step).

**What this enables (untested at submission time):**

- PPO action stream becomes the deployable trajectory — each action is a position target the LX-16A interpolates to over the 16 ms ctrl interval
- Bang-bang is impossible: the sim physically rate-limits the action stream before it ever reaches the PD
- Deployability gate: if `ppo_actuator_gap.py` reports saturation $< 30\%$ AND replay ratio $> 50\%$, the trace ports to hardware via `LX16A.move(angle, time=16)` calls

**CMA gaits unaffected** — they don't go through gym_env, and their slow phase times already

# Where We Land — Submission Snapshot

**What deploys to hardware today:**

- **Mammalian CMA gait** (`best_gait.json`, +821 reward, gen 2500)
- **X-config CMA gait** (`cma_xconfig.json`, +680 reward, gen ∼300)
- Both load directly via `gait_controller.py --gait <file>`

**What we learned that the slides don't fully convey:**

- Reward shaping is a search problem within the search problem
- Seed variance dominates ablation effects until you commit to 5+ seeds per cell
- Sim-to-real is not one gap, it's a stack: friction, then actuator-model, then sensor model

**What's next, ready but unrun:**

- PPO with `--slew-rate 125 --kp 20` — the hypothesis is that the velocity-slew actuator removes the bang-bang exploit and makes PPO traces deployable
- Bench friction calibration (the same step-response idea, but for $\mu$)

*Optimizing the gait was easy compared to learning what we were optimizing for.*